

# Cloud and Edge Computing

**Sami Yangui, Ph.D**  
**A. Prof and CNRS LAAS Researcher**

- Related and Relevant Concepts
- Virtualization
- Cloud Computing
- Autonomic Computing
- Edge Computing

# Part 1: Relevant and Related Concepts

# DISTRIBUTED COMPUTING

- Distributed Computing consists of several and *remote* components that are *connected* with each others *within a single environment* and coordinate their actions via *messages to reach the same objective*.
- SOA, cloud computing and P2P systems are among the examples

# PERVASIVE COMPUTING

- Pervasive (a.k.a ubiquitous) computing refers to the calculation and/or execution that can occur using *any device*, in *any location*, at *any time* and in *any format*
- Pervasive Computing is relevant to wide range of computing models such as *mobile computing*, *fog computing*, artificial intelligence
- Mobile phones, GPS systems, IoT devices are among the examples

# UTILITY COMPUTING

- Utility Computing is a *business model* in which service *providers* own, operate and manage computing facilities and resources, as well as, the end-users access to them
- Novel and sophisticated metering and *billing* system
- *Cooperation* and *federation* between providers needs to be supported

# SERVICE-ORIENTED COMPUTING

- SOC is the paradigm for *Distributed Computing* to enable *application* networks to be built using *services as fundamental elements* for developing applications.
- SOC manifesto came up with 6 core values:
  - Business value
  - Strategic goals
  - Flexibility
  - Intrinsic inter-operability
  - Evolutionary refinement
  - Shared services

# BEYOND SERVICE-ORIENTED ARCHITECTURE

- Web 2.0 (the current state of online technology)
  - Greater collaboration among Internet users (user-generated content, collaborative efforts)
  - The emergence of content providers
  - The introduction of the social nature (social networking, social curation)



# BEYOND SERVICE-ORIENTED ARCHITECTURE (CONT.)

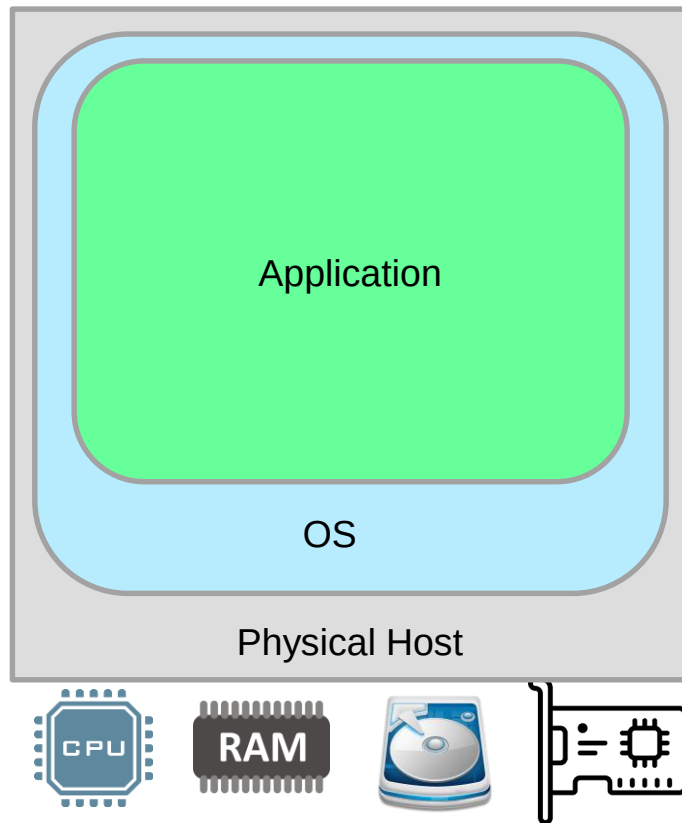
- Microservices (A “kind of” service-oriented architecture)
  - Strongly based on DevOps
  - Technology and communication agnostic protocols
  - Composition “outside the box”

## Part 2: Virtualization

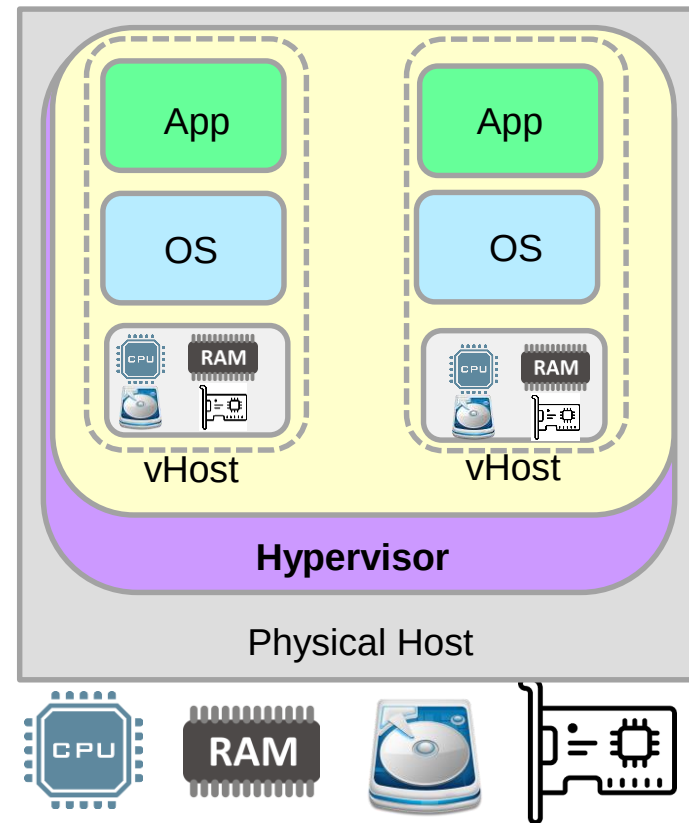
- Enables running *multiple* OS on a *single* physical system
- Shares the underlying hardware resources
- Provides multiple execution environments
  - Hardware and/or software partitioning
  - Partial/complete machine simulation/emulation
- Ensures *isolation*/separation of the resources

# VIRTUALIZATION : DEFINITION (CONT.)

- Without virtualization



- With virtualization

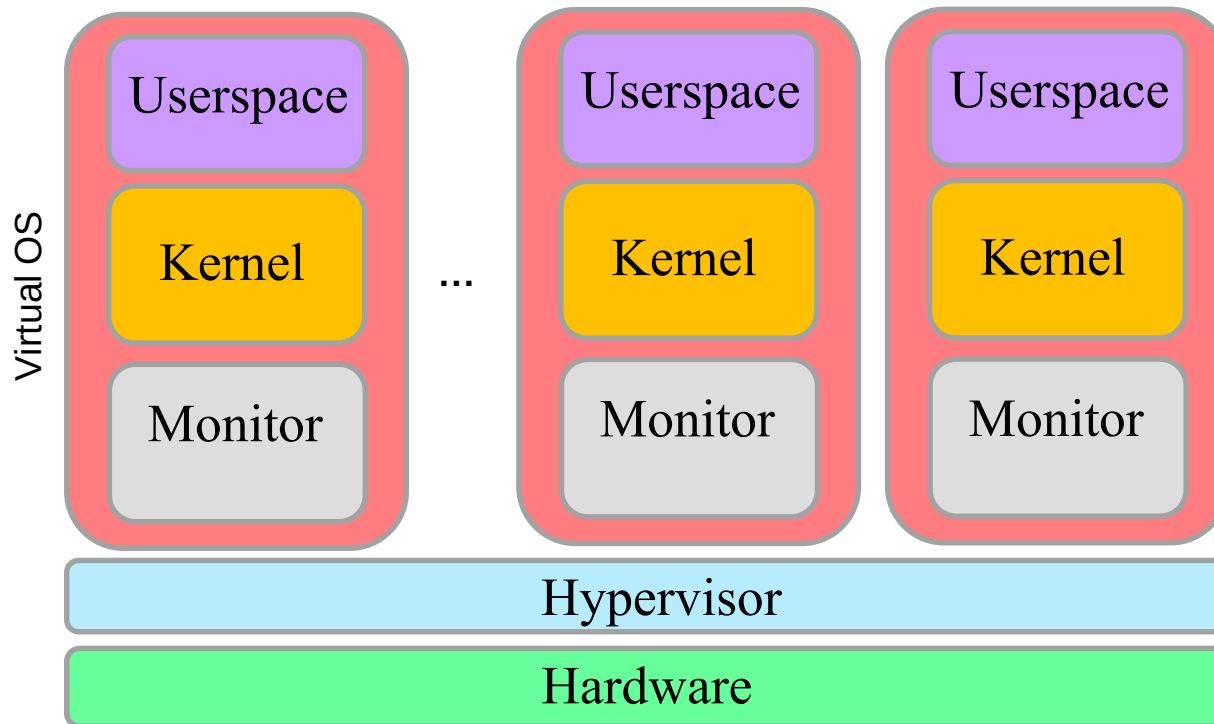


# HYPERVISORS: DEFINITION

- A hypervisor is a *hardware virtualization technique* that allows multiple OS to run on a single host system *at the same time*
- A hypervisor enables the guest OS to *share the hardware of the host*, such that each OS appears to have its own processor, memory and other hardware resources
- A hypervisor is also a *software* (a.k.a Virtual Machine Manager) that allows creating, managing and/or terminating *Virtual Machines (VM)*

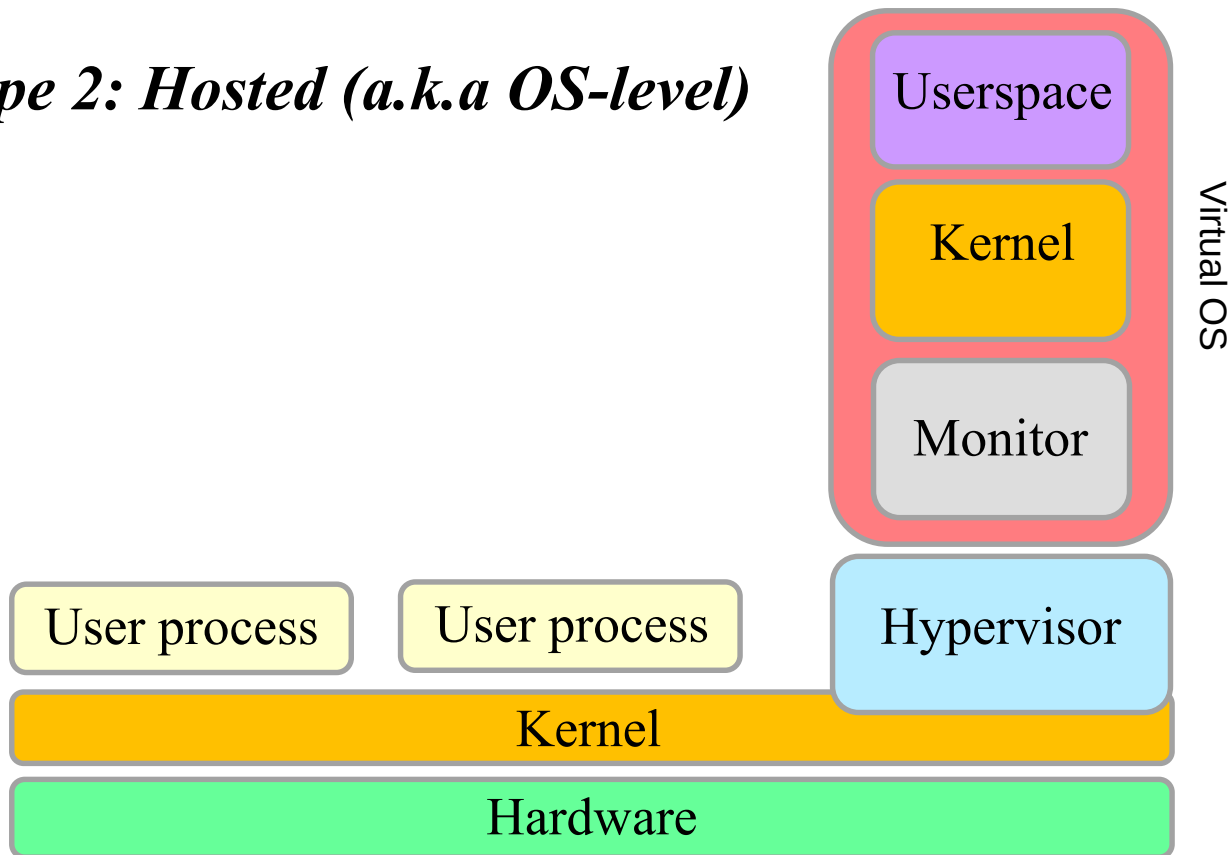
# HYPERVISORS: ARCHITECTURES

- There are 2 types of architectures
  - *Type 1: Bare-metal (a.k.a Native or Hardware-level)*

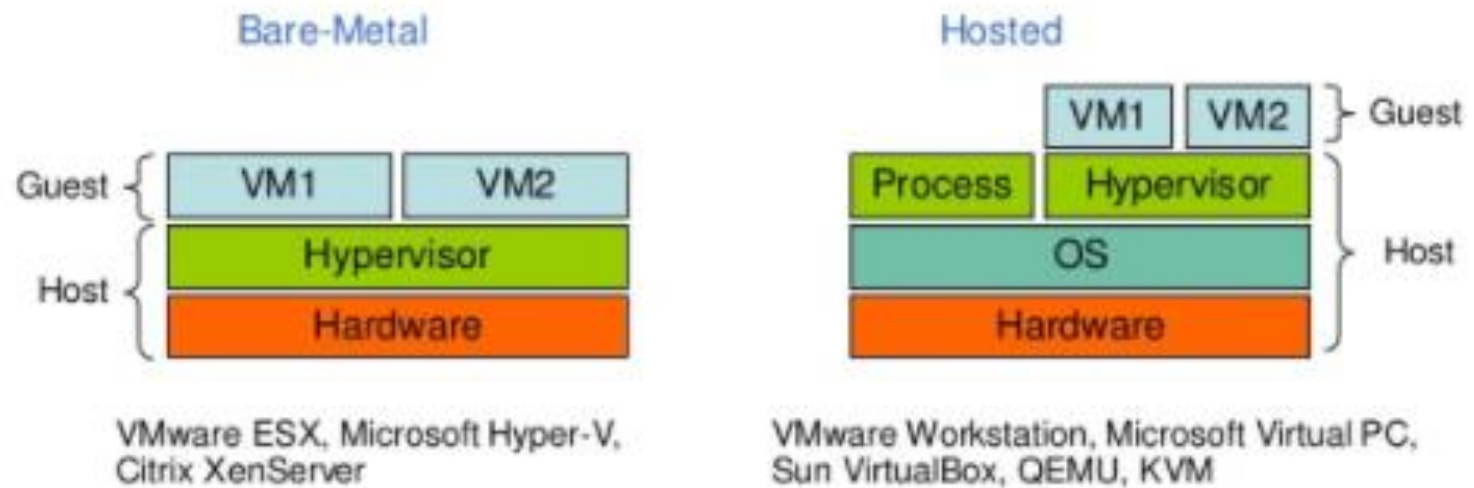


# HYPERVISORS: ARCHITECTURES (CONT.)

## – *Type 2: Hosted (a.k.a OS-level)*



# HYPERVISORS: ARCHITECTURES (CONT.)



Source: Virtualization overview, white paper by VMware



# PARAVIRTUALIZATION

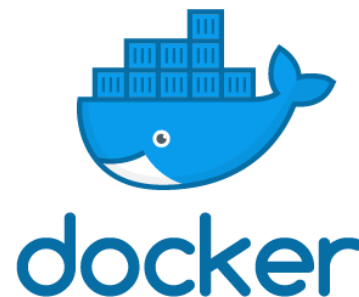
- *Application Programming Interface (API)* is exposed by the hypervisor to the Guest OS for the hardware use
- Guests interact with the hypervisor at a higher-level of abstraction
  - Instead of supplying the specifics of how to use the hardware, software provides general requests to the hypervisor



# CONTAINERS (OS VIRTUALIZATION)

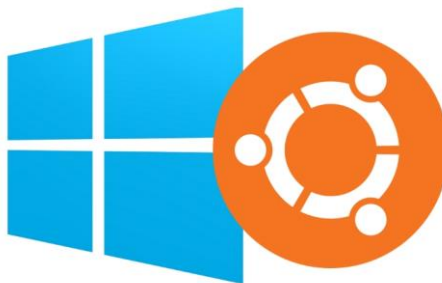
- Instead of virtualizing the hardware, run *multiple virtual instances of the same OS* over a single hardware
- Containers provide *isolation* between processes, appear as separate OS.
- Single kernel means *very low overhead* (1..3\*%) compared to standalone server.

\* Source: Docker Corp Experiments



# VM TEMPLATES

- A VM template is the VM's pattern that consists of virtual disks, OS and related system settings
  - *Concrete* VMs are instances of *abstract* templates
  - Templates save time and avoid errors when configuring settings (VMs consistency)
  - Templates can also be used as long-term in-place backups of VMs



# VIRTUAL APPLIANCES

- A Virtual Appliance (VA) is a pre-configured VM template that is *pre-packaged* with the necessary software to serve its intended purpose
  - VAs are aimed to eliminate the installation, configuration and maintenance costs associated with complex stacks of software



# VM IMAGES

- A VM image is a ***persistent snapshot*** that consists of a VM template (eventually, a VA) + prospective installed software + execution context

## Part 3: Cloud Computing

*There is no definition yet...but, there is a common consensus*

- It enables *ubiquitous*, convenient, *on-demand* network access to a *shared pool* of *configurable* computing resources (e.g. networks, servers, storage resources, services and applications)
- It provides mechanisms to swiftly *provision* these resources and release them with *minimal management effort*.

Source: The NIST definition, 2011

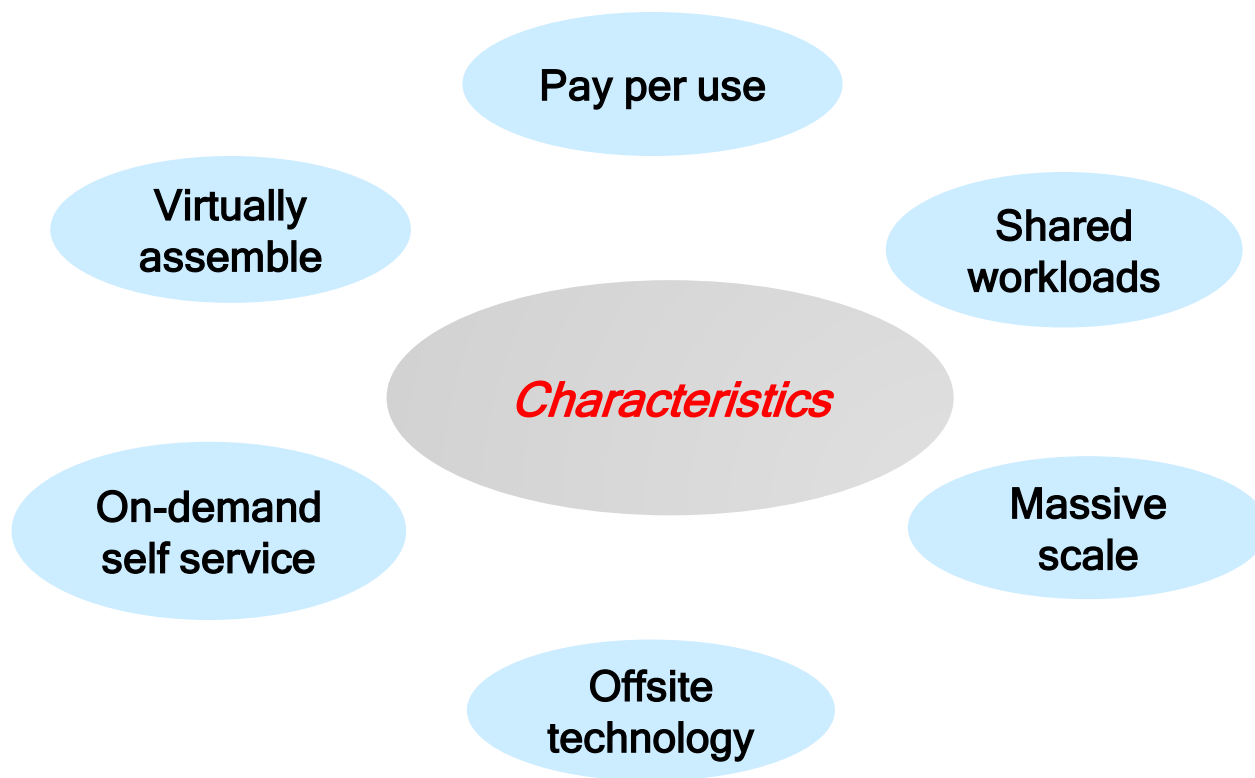
- Implements a large pool of easily *usable and accessible virtualized resources* (such as hardware, development platforms and/or services).
- Handles resources that can be *dynamically reconfigured* to adjust to a *variable load*, allowing also for an *optimum resources utilization*.
- Uses the *pay-per-use* (a.k.a *pay-as-you-go* or *à la carte*) model

Source: Vaquero et al., 2009



- A better approach than a definition ➡ Identify the distinctive characteristics
- Examples of characteristics as per Ambrust et al., 2010
  - Appearance of *infinite computing resources* available *on-demand*
  - Elimination of an upfront commitment by end-users
  - Ability to pay *à-la-carte*

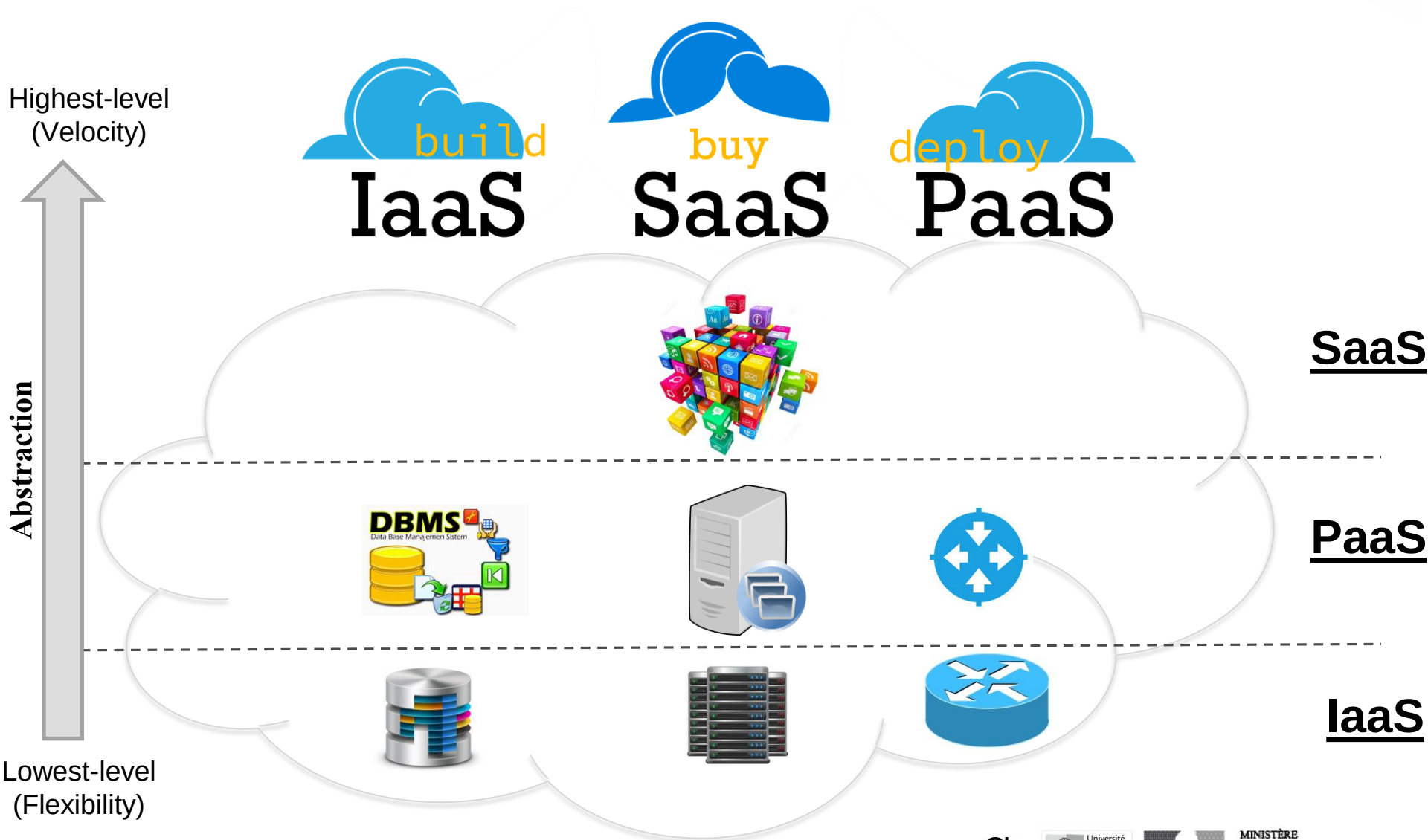
# CHARACTERISTICS



# KEY ENABLING TECHNOLOGIES

- Virtualization
- Web services – Web 2.0
  - SOAP-based (almost dead)
  - REST-based
- Utility Computing

# SERVICE DELIVERY MODELS (XAAS)



SaaS

PaaS

IaaS

# SERVICE DELIVERY MODELS (XAAS)



salesforce

webFilings™

Sample  
of SaaS

CF  
Pivotal Cloud Foundry®

Jelastic

heroku

Google Cloud Platform

OPENS SHIFT®  
by Red Hat®

Sample  
of PaaS



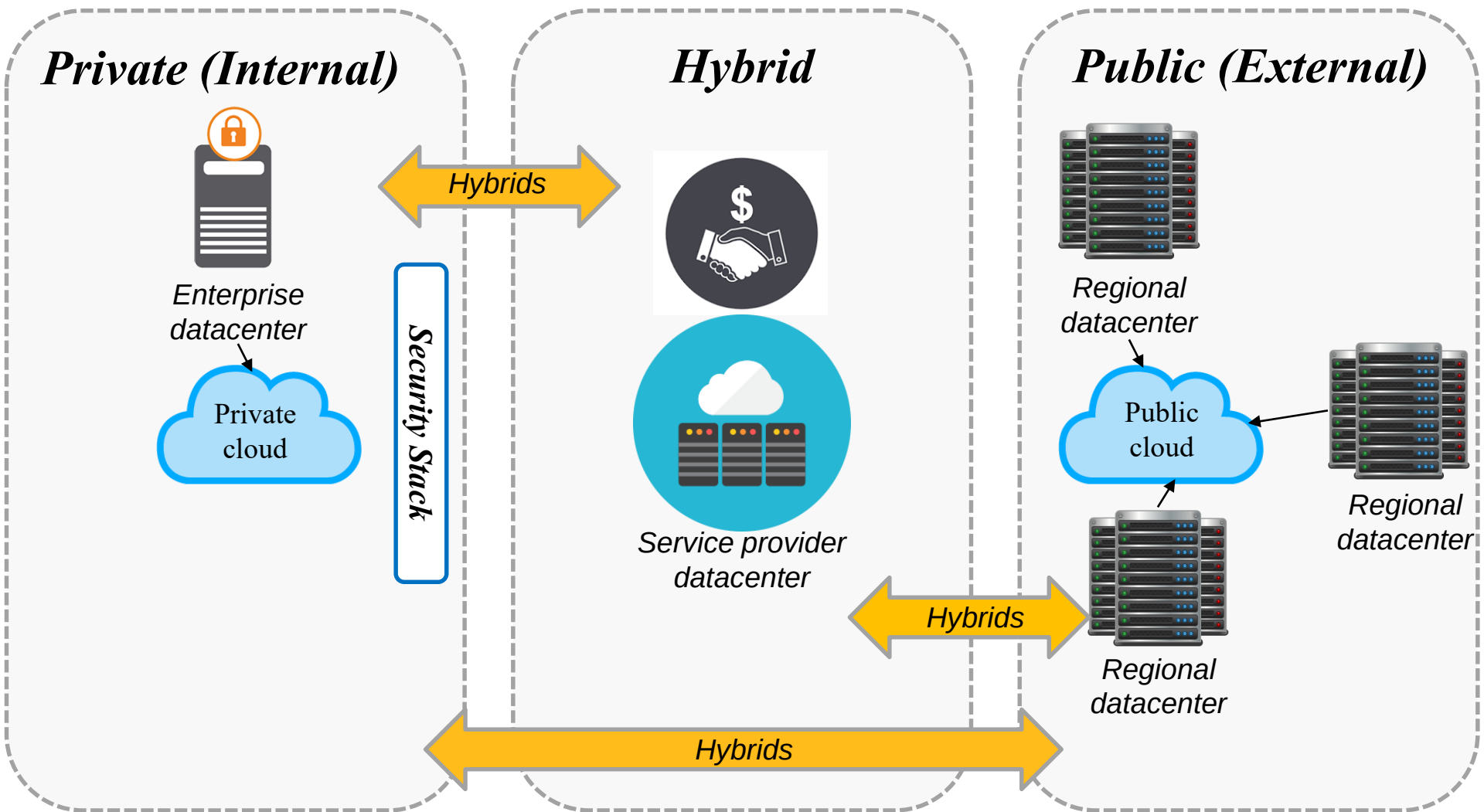
Google Compute Engine

rackspace®

IBMSmartCloud

Sample  
of IaaS

# SERVICE DEPLOYMENT MODELS



# SERVICE DEPLOYMENT MODELS (CONT.)

| Public Cloud                                                   | Private Cloud                                        |
|----------------------------------------------------------------|------------------------------------------------------|
| Owned and managed by service providers                         | Owned solely by an enterprise                        |
| Made available to the general public or a large industry group | Operated for an organization or a third-party        |
|                                                                | Limits access to enterprise and partner network      |
|                                                                | Retains high degree of control, security and privacy |

# SERVICE DEPLOYMENT MODELS (CONT.)

## Community Cloud

Shared infrastructure by several organizations which have shared concerns

Maybe managed by the organizations or a third-party

Costs are spread over fewer users than a public cloud but more than a single tenant

## Hybrid Cloud

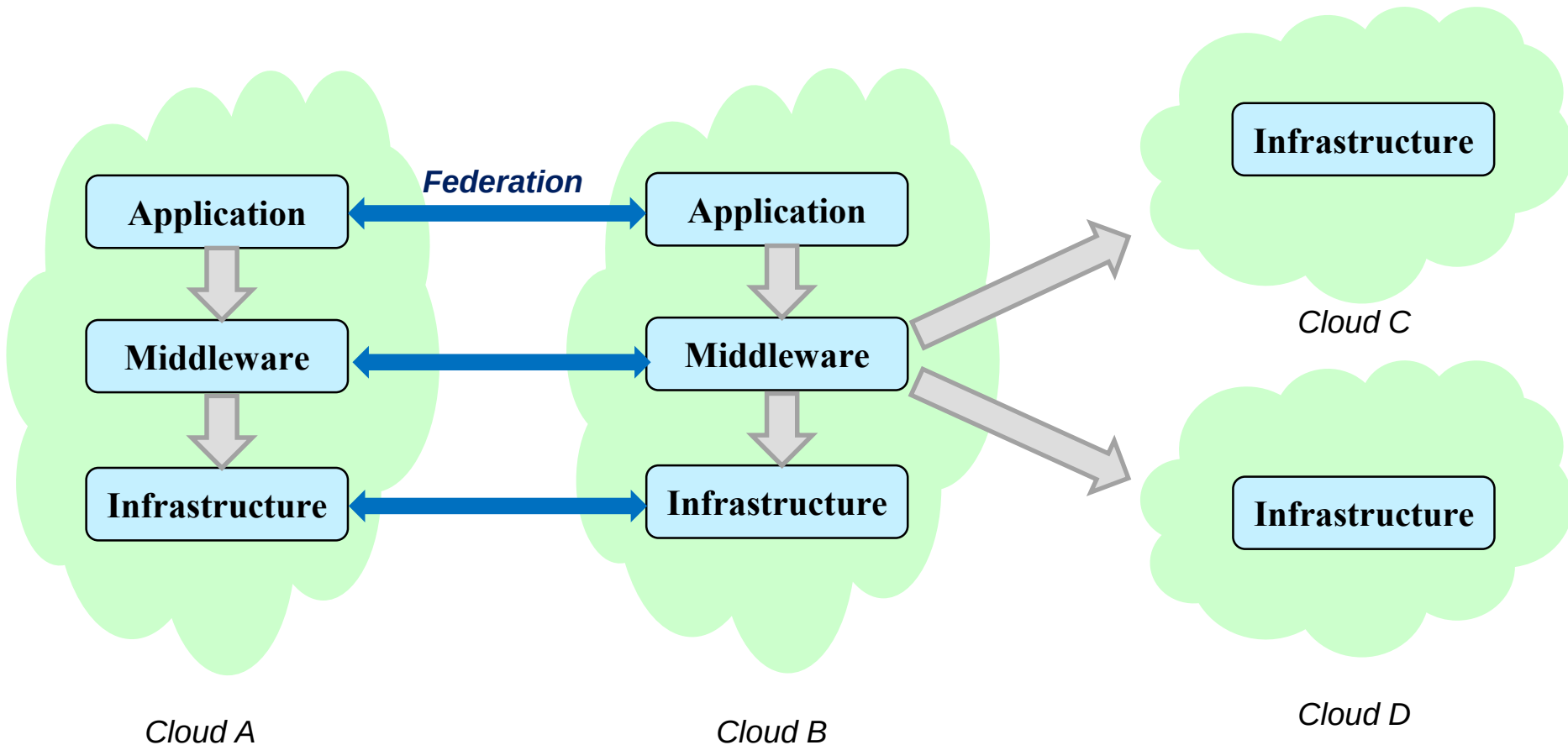
Composition of two or more clouds (private, community or public) bound together by **standardized or proprietary technologies** that enable **data and application portability**



# CLOUD FEDERATION AND COOPERATION

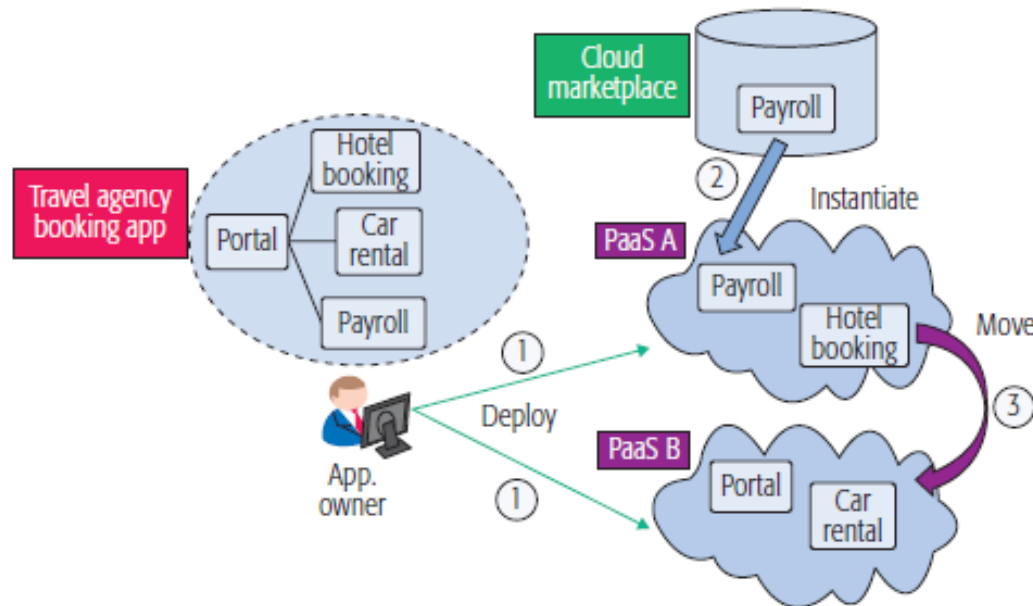
- Interconnects cloud-based environments of two or more service providers to dynamically accommodate workload and spikes
- Intervenes at IaaS, as well as, at PaaS levels
- Hides its complexity to end-users (including SaaS)
- Requires services and applications portability for optimal operating

# CLOUD FEDERATION AND COOPERATION (CONT.)



# CLOUD PORTABILITY

- It is the ability to move applications, services and data from one cloud environment to another with minimal transformation and integration efforts.



Source: Yangui et al., 2016

# Part 4: OpenStack Cloud Software

# WHAT IS IT?



- OpenStack is an IaaS manager and a cloud operating system
- It takes
  - Resources such as compute, storage and network
  - Technologies such as SOA and virtualization
- It provides
  - Resources control and management at datacenter level

# OPENSTACK CONSORTIUM

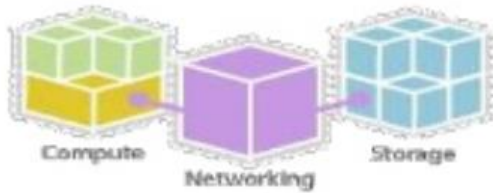


**openstack™**  
CLOUD SOFTWARE

- OpenStack is a consortium: A community of open source developers, participating organizations and users who are building and using the product



# OPENSTACK PROJECT

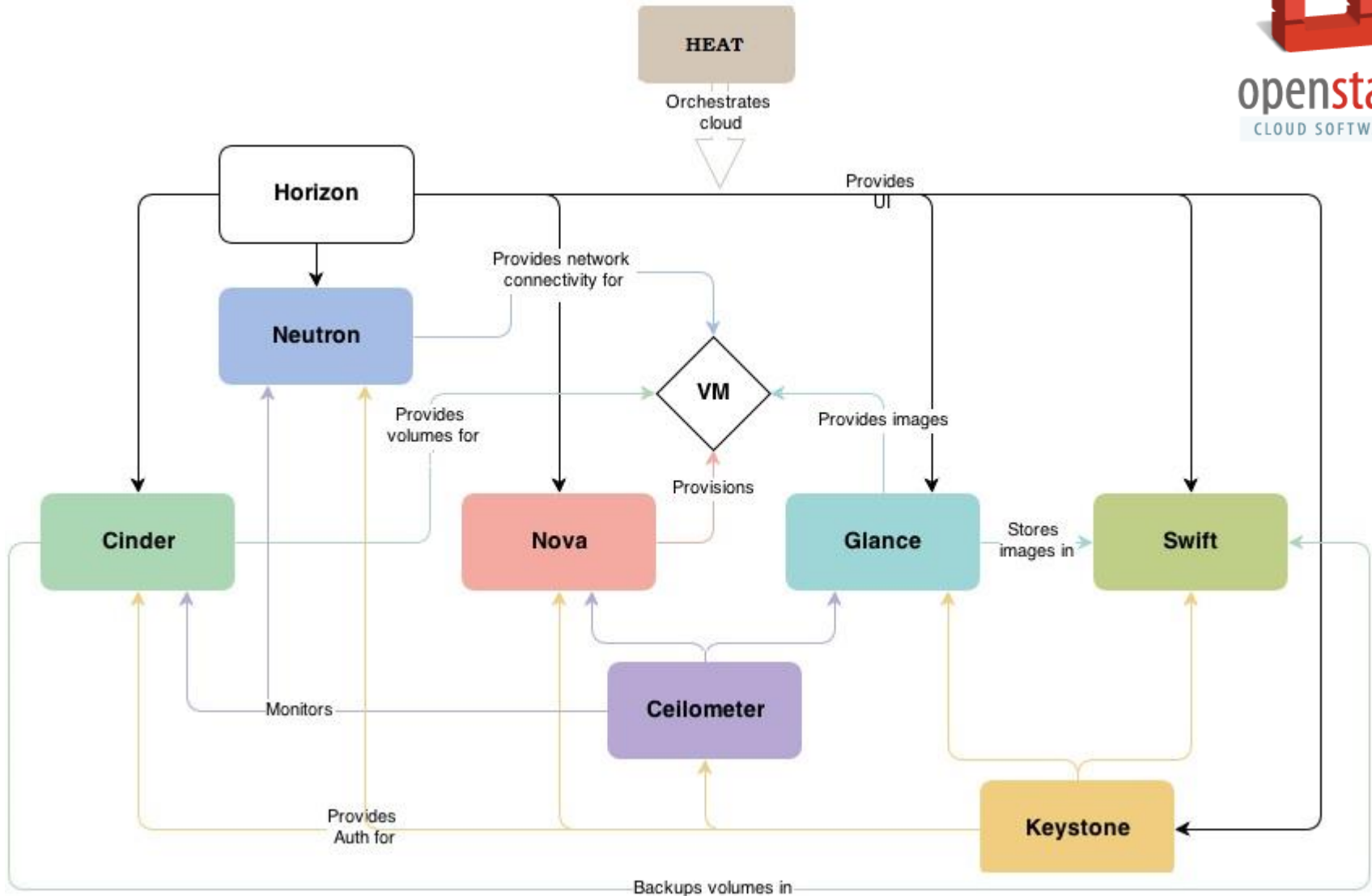


- **OpenStack Compute (Nova):** Provision and manage large networks of virtual machines
- **OpenStack Open Store (Swift):** Create petabytes of reliable storage using standard services
- **OpenStack Image Service (Glance):** Catalog and manage large libraries of server images
- **OpenStack Quantum Service:** Provide Network as-a-Service to compute
- **Other components:** Dashboard, Authentication (keystone), CLI, etc.

# OPENSTACK PROJECT (CONT.)



**openstack™**  
CLOUD SOFTWARE





# OPENSTACK CAPABILITIES



- Builds public and private clouds
  - Flexibility and various use cases
  - Mix and match components
  - Kit of powerful tools
- Enables multi-tenancy
  - Quota for different users
  - Users can be associated with multi-tenants

# OPENSTACK CAPABILITIES

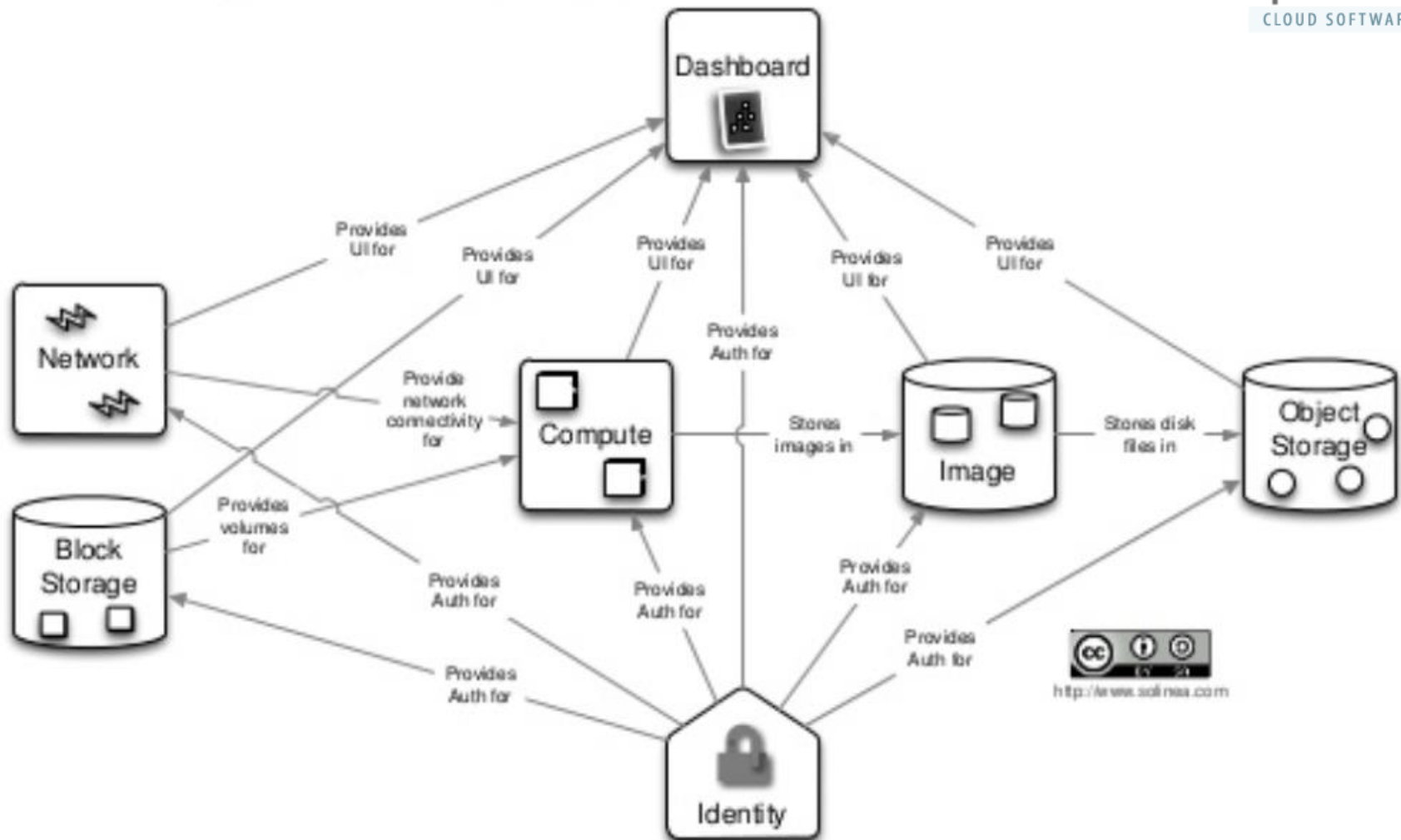


- Provides virtual machines on-demand
  - Self service provisioning
  - Snapshotting capability
- Handles storage volumes
  - Block storage for VM images
  - Object storage for VM images and arbitrary files

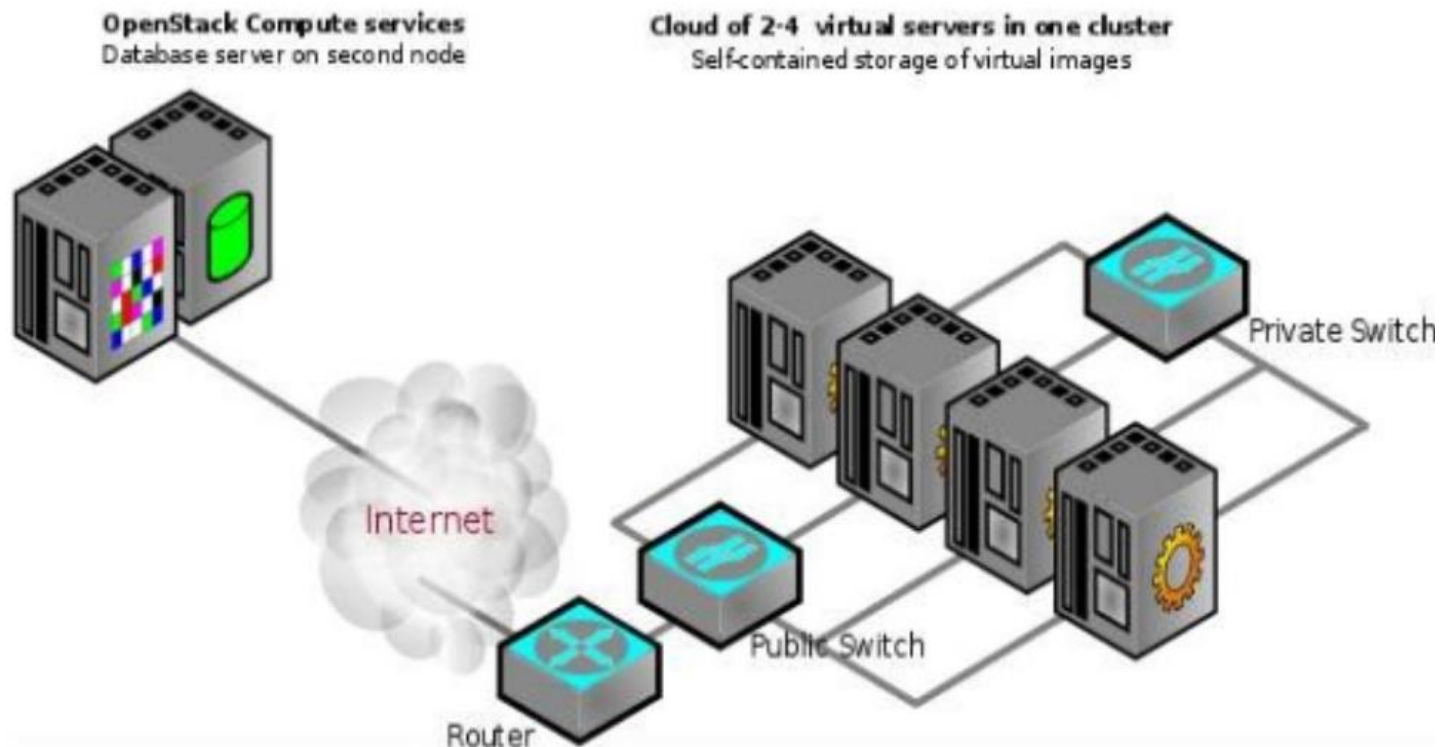
# OPENSTACK ARCHITECTURE



openstack™  
CLOUD SOFTWARE



# SAMPLE DEPLOYMENT



Source: OpenStack documentation

# Part 5: Autonomic Computing

# DEFINITION



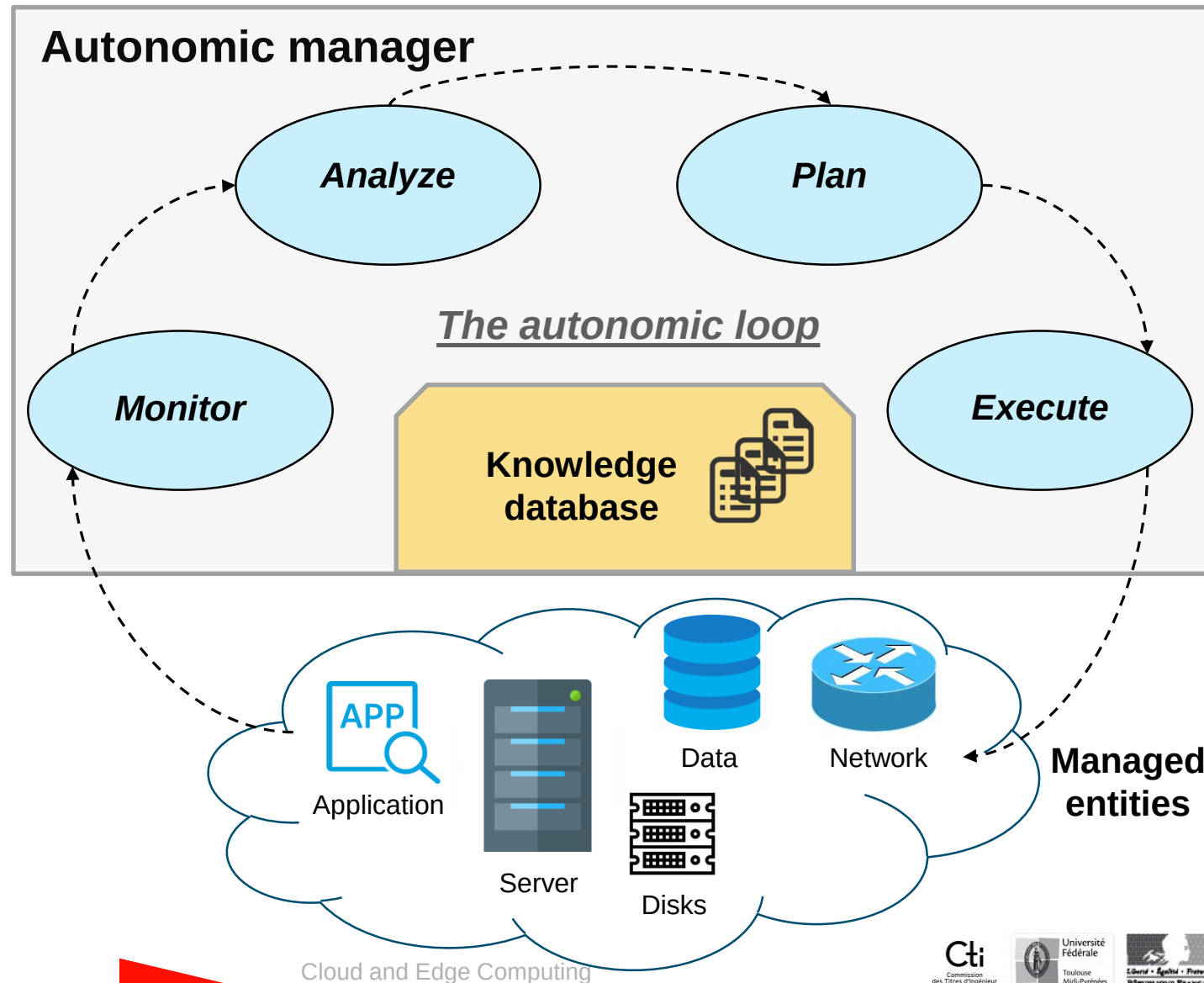
- Approach to enable systems to manage *themselves* without direct human intervention
- Inspired by human autonomic nervous system which controls important bodily functions without conscious intervention
- A methodology for reducing the complexity of maintaining distributed systems

Source: Autonomic computing, IBM, 2001

# FUNDAMENTAL PRINCIPLE

Autonomic computing is an application of artificial intelligence for the purpose of *self-determination*

# OPERATING PRINCIPLE





# CHARACTERISTICS

- Self-Healing
- Self-Configuring
- Self-Protecting
- Self-Optimizing

|                        |                         |
|------------------------|-------------------------|
| <b>Self-Healing</b>    | <b>Self-Configuring</b> |
| <b>Self-Protecting</b> | <b>Self-Optimizing</b>  |

# CHARACTERISTICS (CONT.)

|                        |                         |
|------------------------|-------------------------|
| <b>Self-Healing</b>    | <b>Self-Configuring</b> |
| <b>Self-Protecting</b> | <b>Self-Optimizing</b>  |

- Self-Healing
  - Discover, diagnose and react to disruptions (without disrupting the service environment)
  - Fault-components should be:
    - Detected
    - Isolated
    - Fixed
    - Reintegrated

# CHARACTERISTICS (CONT.)

|                        |                         |
|------------------------|-------------------------|
| <b>Self-Healing</b>    | <b>Self-Configuring</b> |
| <b>Self-Protecting</b> | <b>Self-Optimizing</b>  |

- Self-Configuration
  - Automated configuration of components and systems follow high-level rules and policies
  - Rest of system adjusts automatically and seamlessly

# CHARACTERISTICS (CONT.)

|                        |                         |
|------------------------|-------------------------|
| <b>Self-Healing</b>    | <b>Self-Configuring</b> |
| <b>Self-Protecting</b> | <b>Self-Optimizing</b>  |

- Self-Protecting
  - Anticipate, detect, identify and protect against attacks from anywhere
  - The related mechanisms should enable:
    - Defining and managing user access to all computing resources
    - Protecting against unauthorized resources access
    - Detecting intrusions and reporting as they occur

# CHARACTERISTICS (CONT.)

|                        |                         |
|------------------------|-------------------------|
| <b>Self-Healing</b>    | <b>Self-Configuring</b> |
| <b>Self-Protecting</b> | <b>Self-Optimizing</b>  |

- Self-Optimizing
  - Monitor and tune resources automatically
    - Support operating in unpredictable environments
    - Efficient maximization of resource utilization without human interference

# Part 6: Fog Computing: Bringing Cloud and IoT to the Edge of the Network

# MAKING THE IOT CLOUD-AWARE

- *Latency-sensitive apps*
- *Huge Data Flows*

 **Fog/Edge Computing ☺**  
Cloud “*Closer to ground*”  
Real time processing

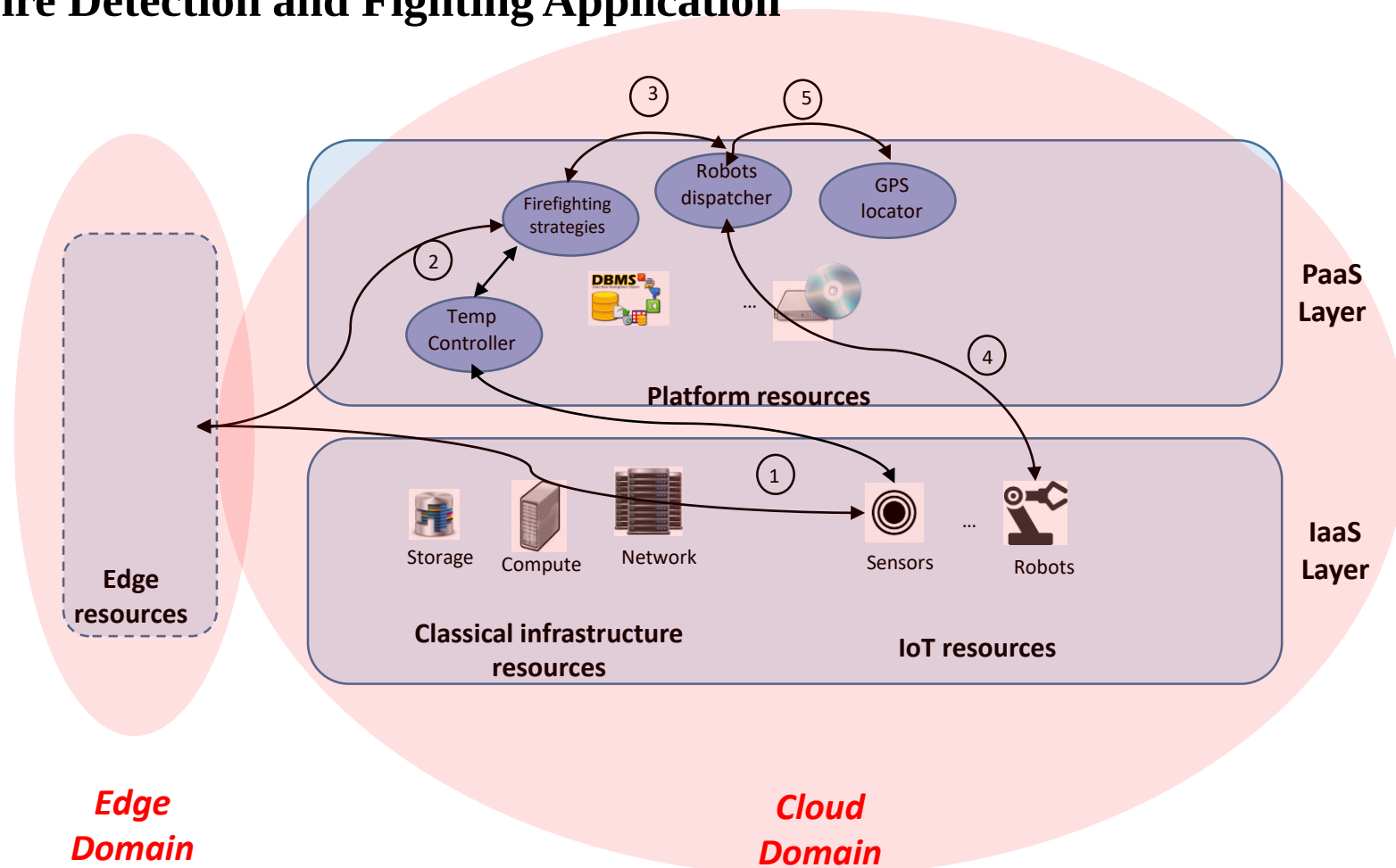
*Unpredictable delays ☹!!*

| IoT                  | Cloud Computing         |
|----------------------|-------------------------|
| Centralized paradigm | Distributed paradigm    |
| Static provisioning  | On-the-fly provisioning |
| No adjustment        | Dynamic adjustment      |

*“Cloudifying” the IoT world !!*

# MOTIVATING USE CASE

## Fire Detection and Fighting Application



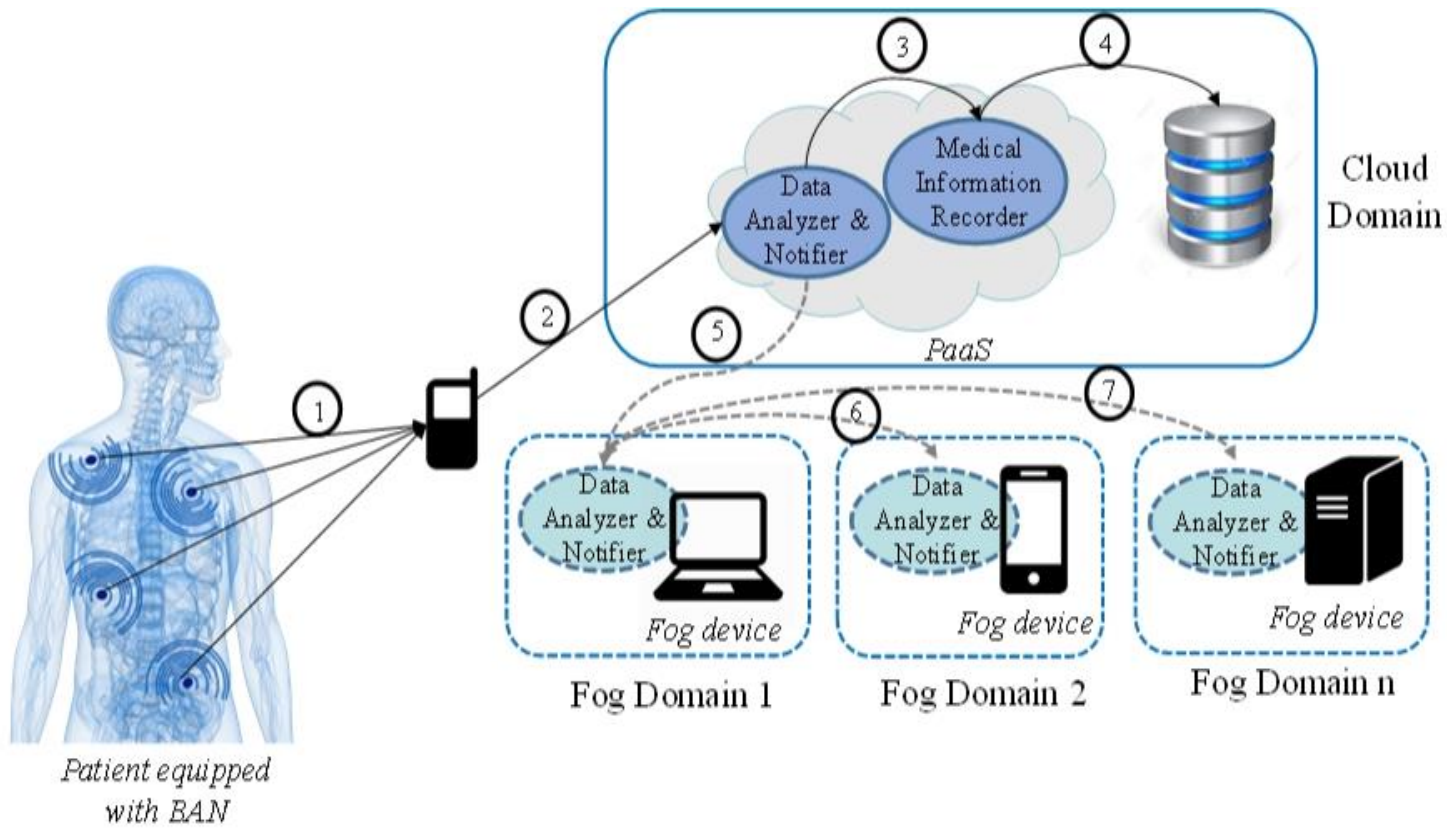
**Edge  
Domain**

**Cloud  
Domain**



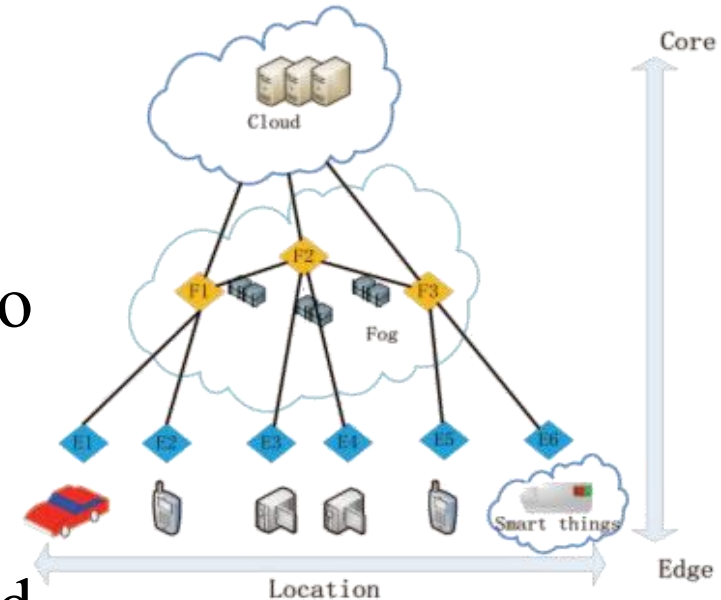
# MOTIVATING USE CASE (CONT.)

## Ambulatory Healthcare Application

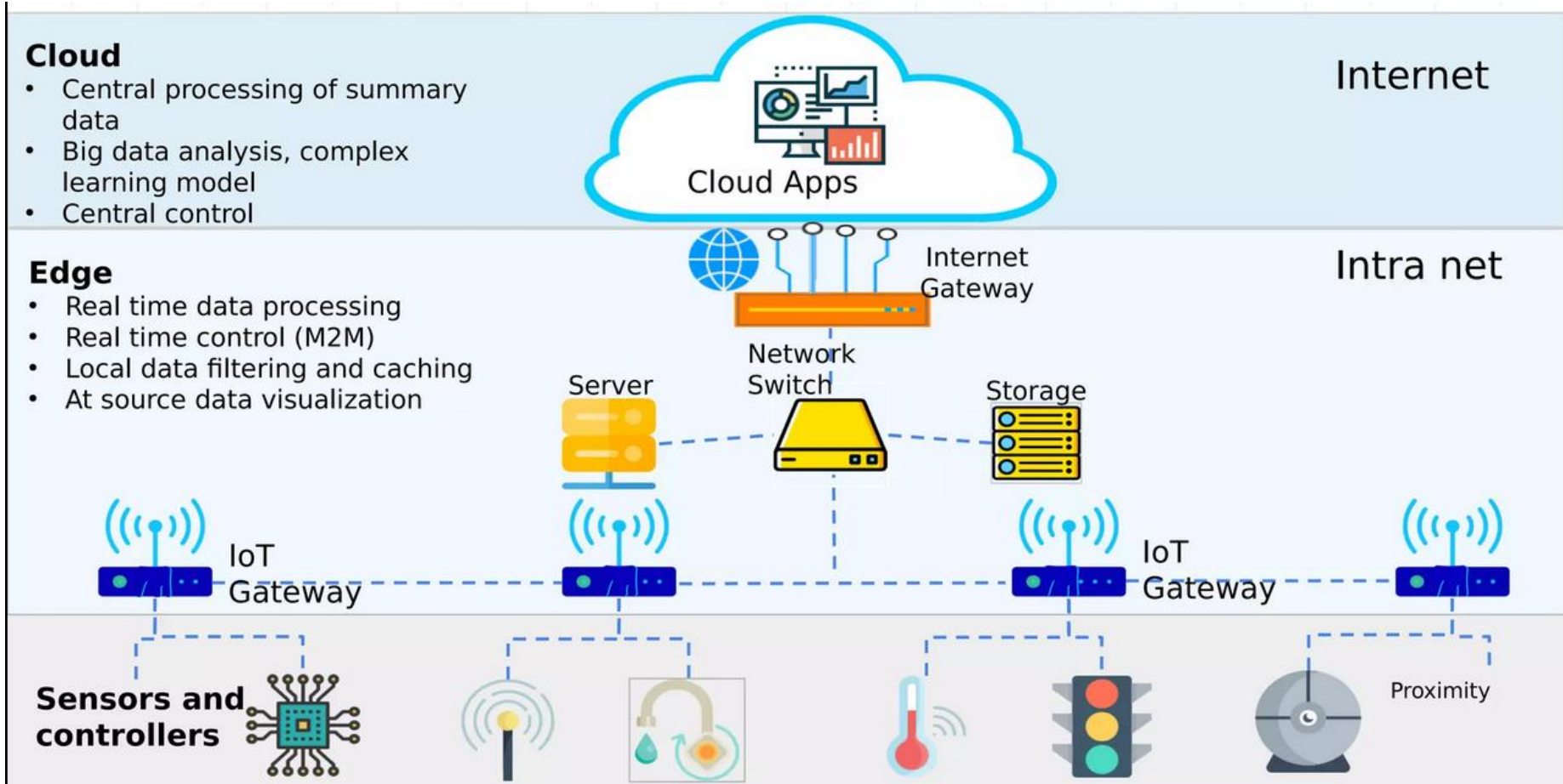


# FOG FACILITIES BETWEEN CLOUD & IOT

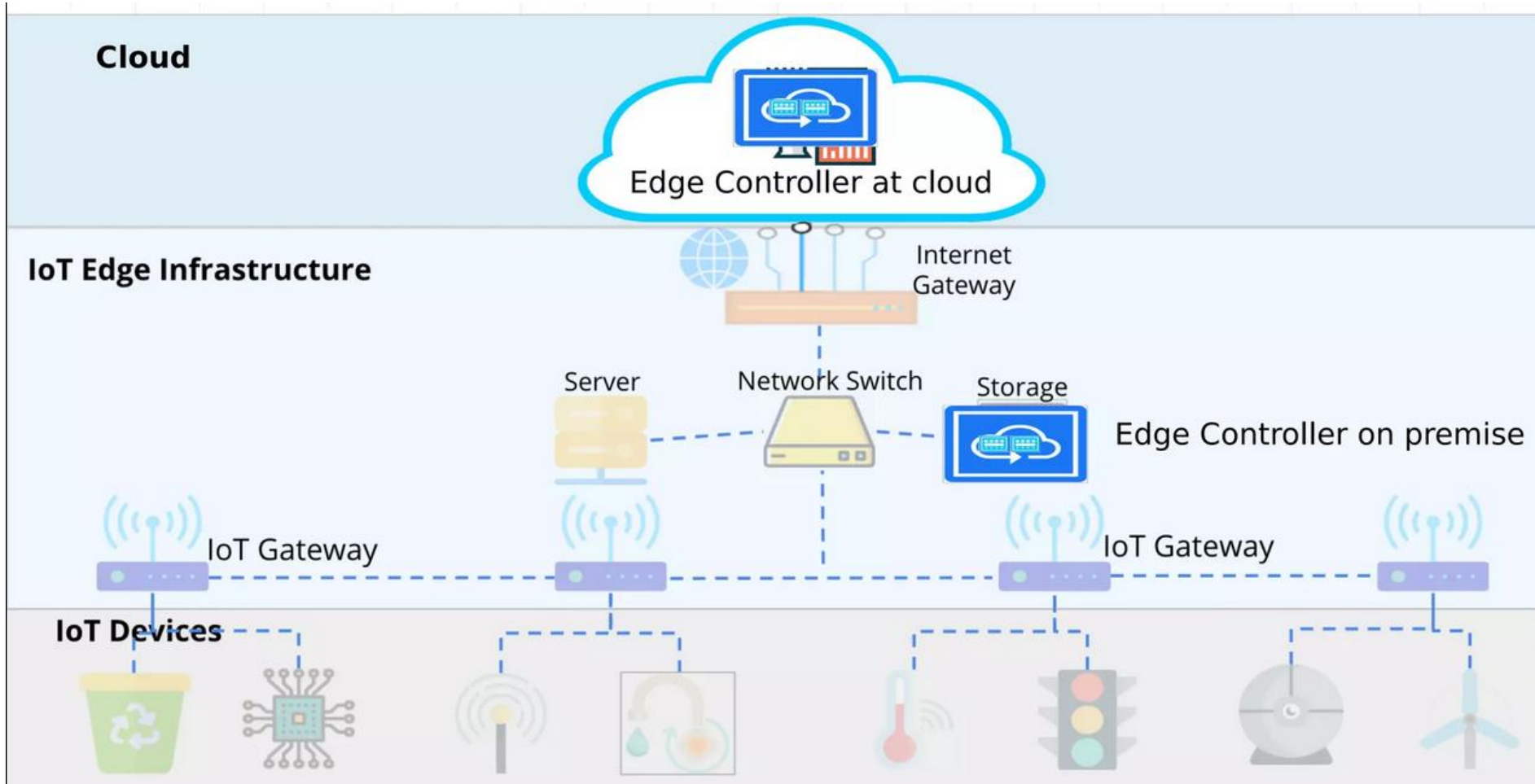
- Fog computing brings cloud close to end-users and data sources.
- IoT systems need both cloud and fog/edge computing.



# SYSTEM OVERVIEW



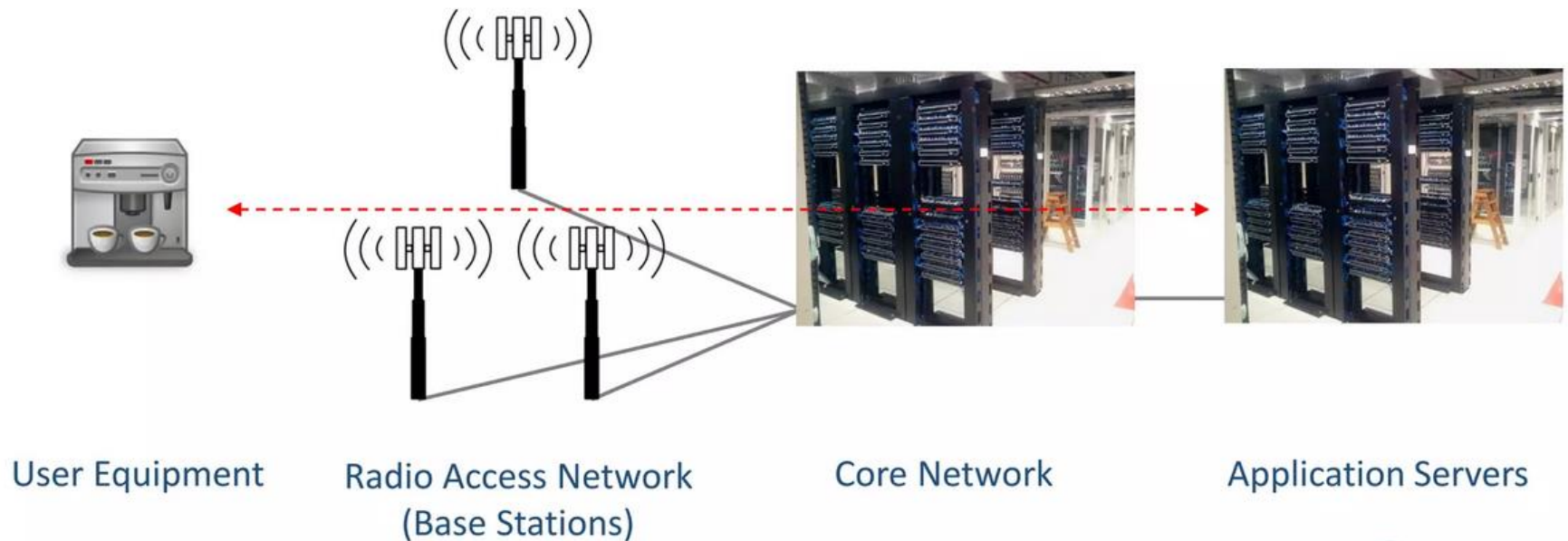
# SYSTEM OVERVIEW



# DATA AND CONTROL PLANES

- Long data path
- Non deterministic QoS

➔ Acceptable for many cloud-based IoT applications



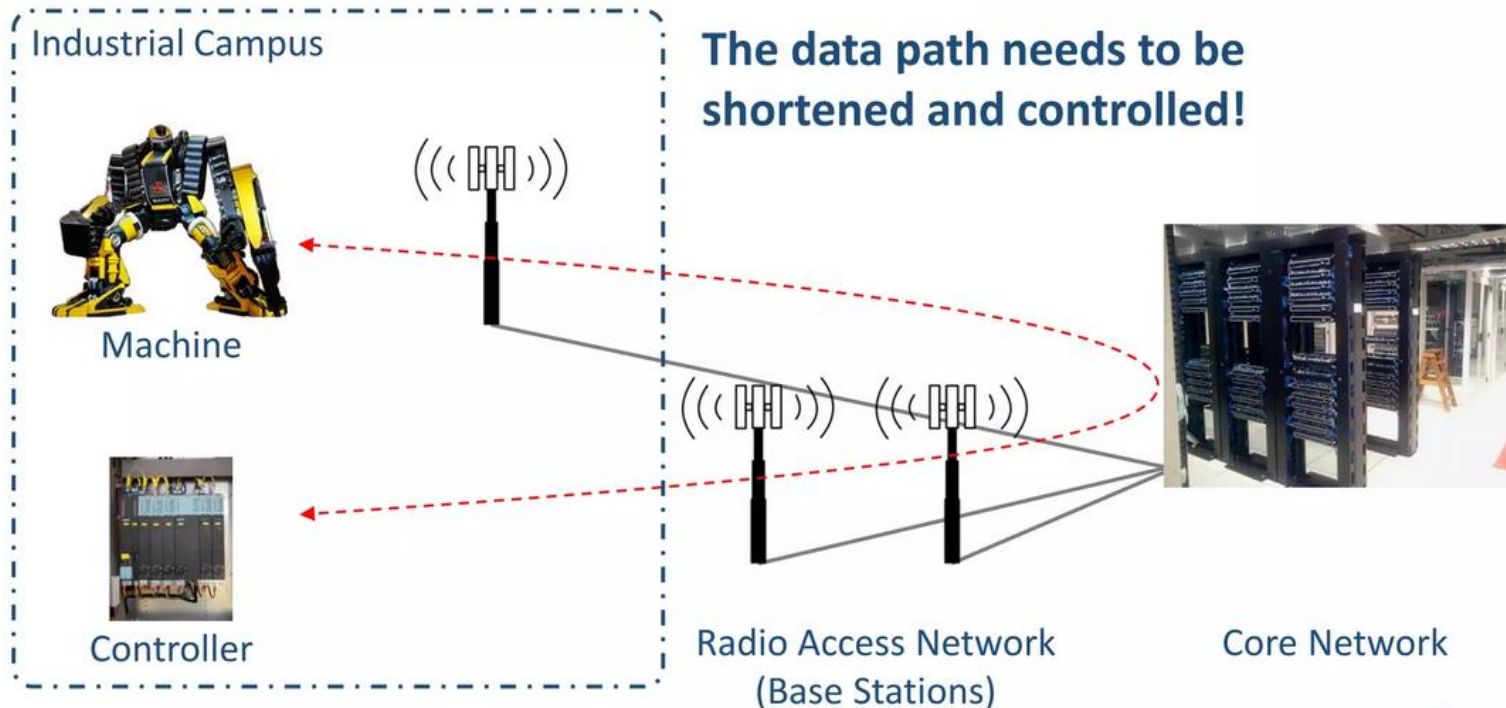


# DATA AND CONTROL PLANES

- Most demanding use cases require licensed spectrum
  - E.g., Industry 4.0 goes wireless
    - Campus-area coverage, deterministic QoS, high availability, seamless mobility
- ➔ Several radio technologies are competing

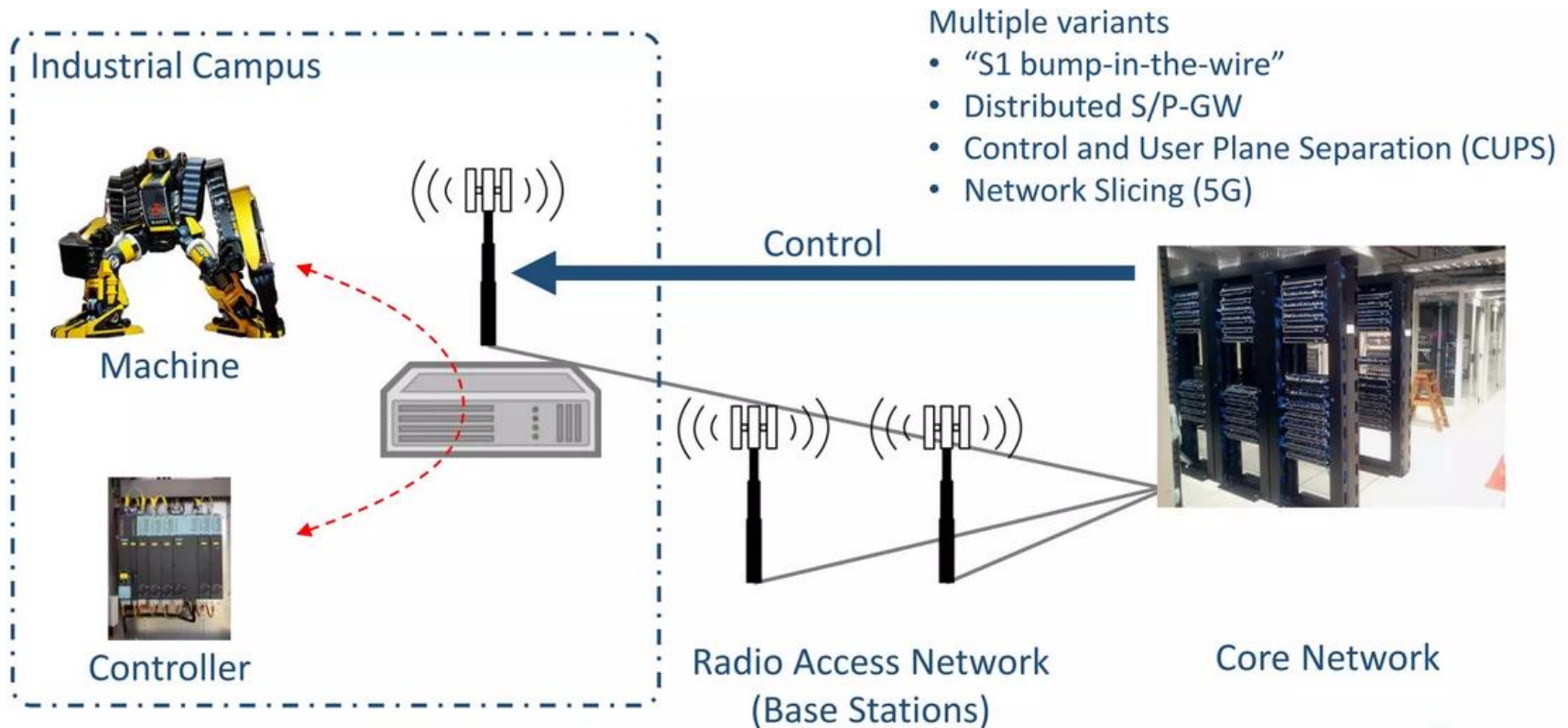


# DATA AND CONTROL PLANES



➡ Relocating the base station does not solve the problem

# DATA AND CONTROL PLANES



➡ Distributing the control plane address the need



# CHARACTERESTICS

- Geographically-distributed
- Massively scalable
- Autonomous
- Contextual and with low latency
- Real-time intercatations
- Heterogeneous

# BENEFITS

- Latency
  - In-line with the next-generation networks
- Cost
  - Reduce communications over the network
  - Rely on third-party entities during operation
- QoS Management
  - Network dynamicity and mobility offer opportunities

# BENEFITS

- Autonomy
  - Compute work on-site
  - Reduced amount of data to be sent
- Data sovereignty
  - Avoid long data journey
  - Data sit close to its sources
  - Securing any sensitive data before sending

# CHALLENGES

- Limited capability
  - Specify the provisioning scope
  - Define the data lifecycle
- Connectivity
  - Accomodate herratic and poor communications
  - Consider what happens when connection is lost
- Security
  - Emphasize proper device management

# COMPAGNIES USING EDGE COMPUTING

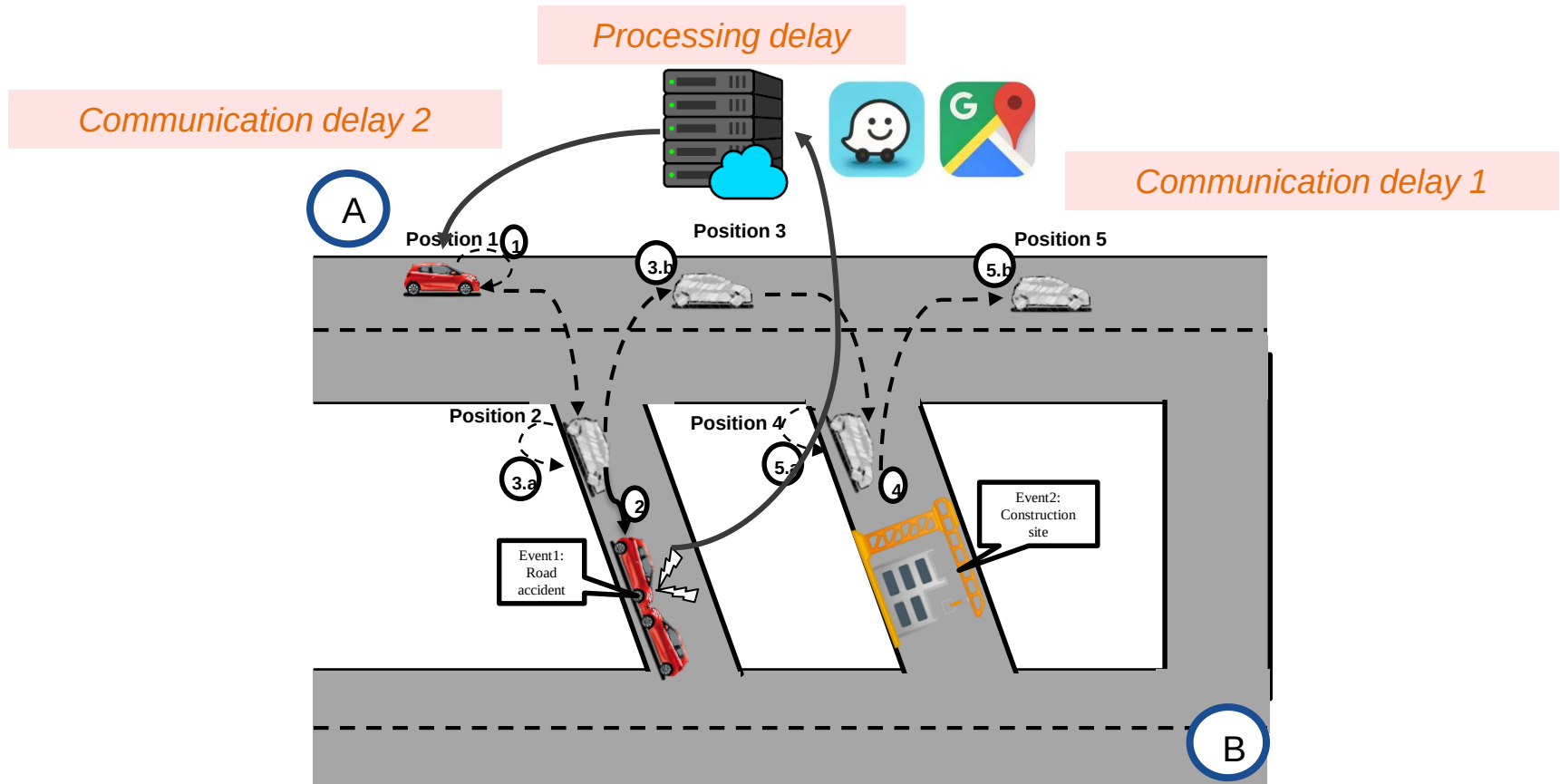


# CHALLENGES & RESEARCH DIRECTIONS

- QoS Management ➡ Autonomic Computing
- Optimal placement
- Control and signaling interfaces
- Dynamic compositions
- IoT devices virtualization

# LAB PROJECT

- Fully-cloud architecture



# LAB PROJECT

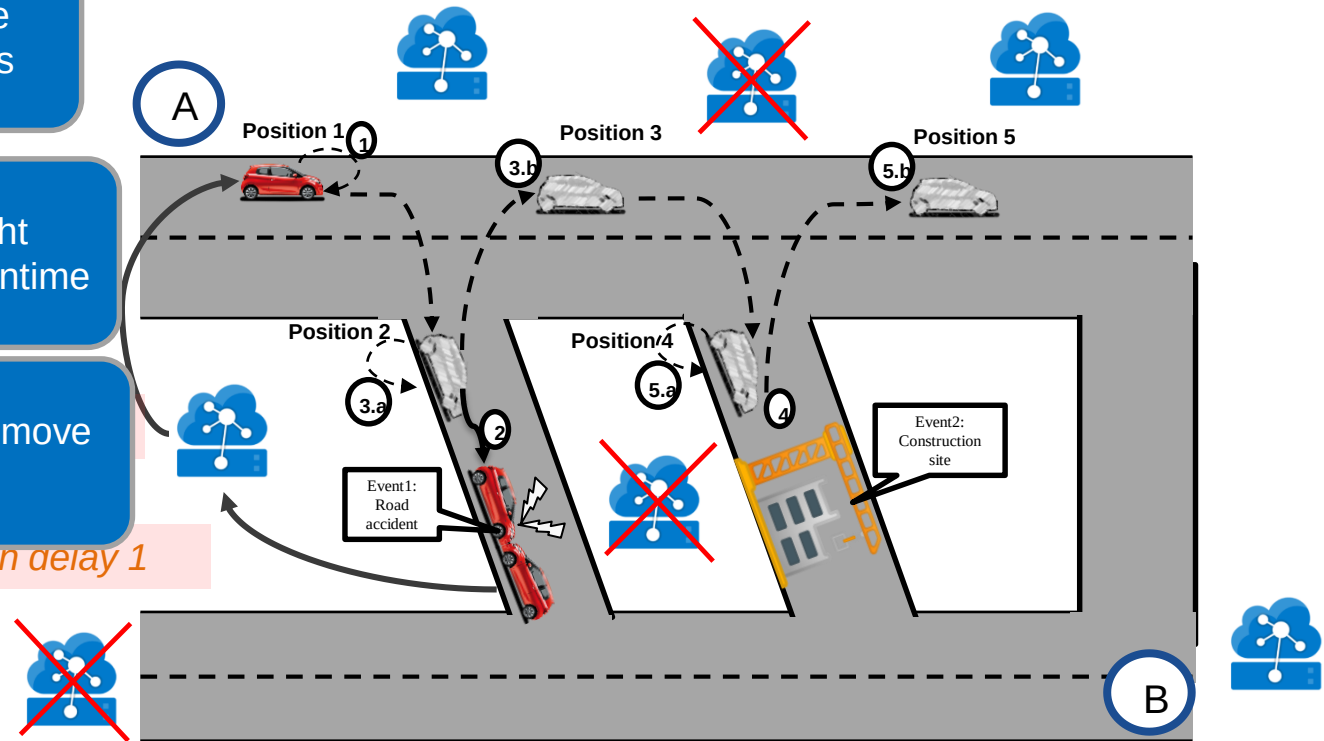
- Hybrid cloud-edge architecture

Edge nodes have random behaviors

1. Edge nodes might appear/disappear at runtime

2. Edge nodes might move at runtime

Communication delay 1





# LAB PROJECT

- Goal: Maintaining service availability during the car trip
- Supporting the network dynamicity and mobility
  - Pods: Implement the network topology
  - Docker: Host and execute the services
  - K8S: Manage clusters and proceeds with service migration